# p8106_hw1_sy3352

Su Yan

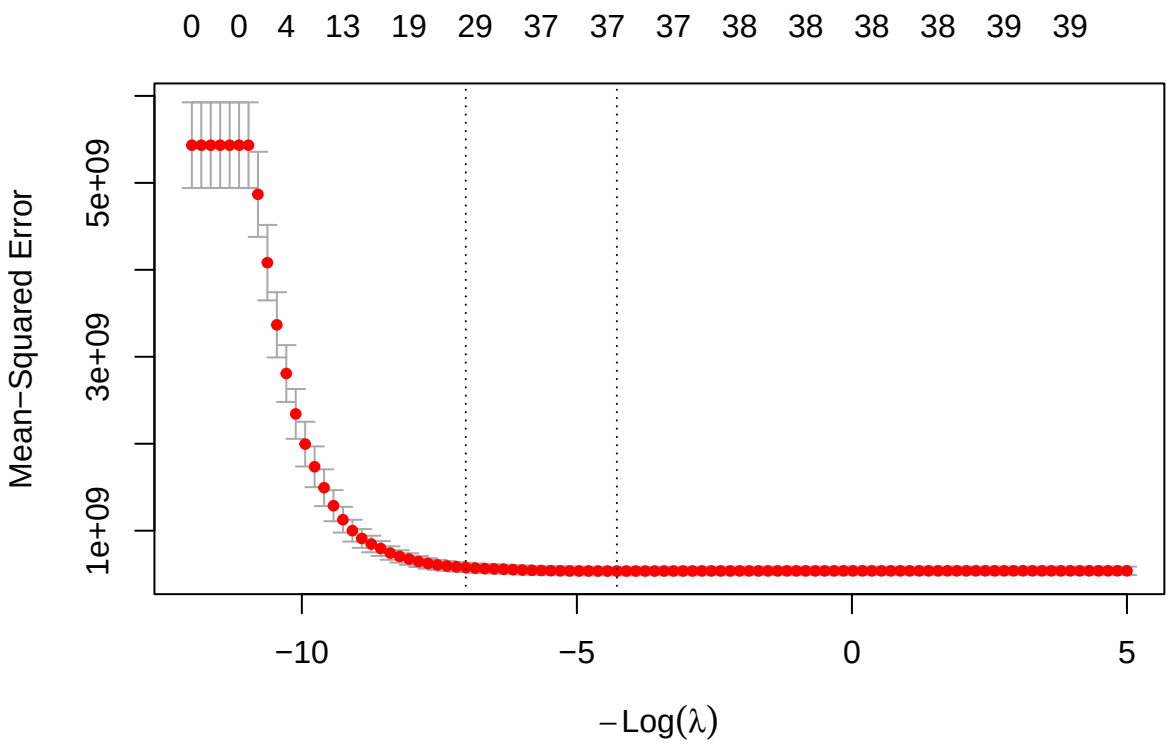
2026-02-24

```
library(glmnet)
library(caret)
library(tidymodels)
library(corrplot)
library(ggplot2)
library(plotmo)
```

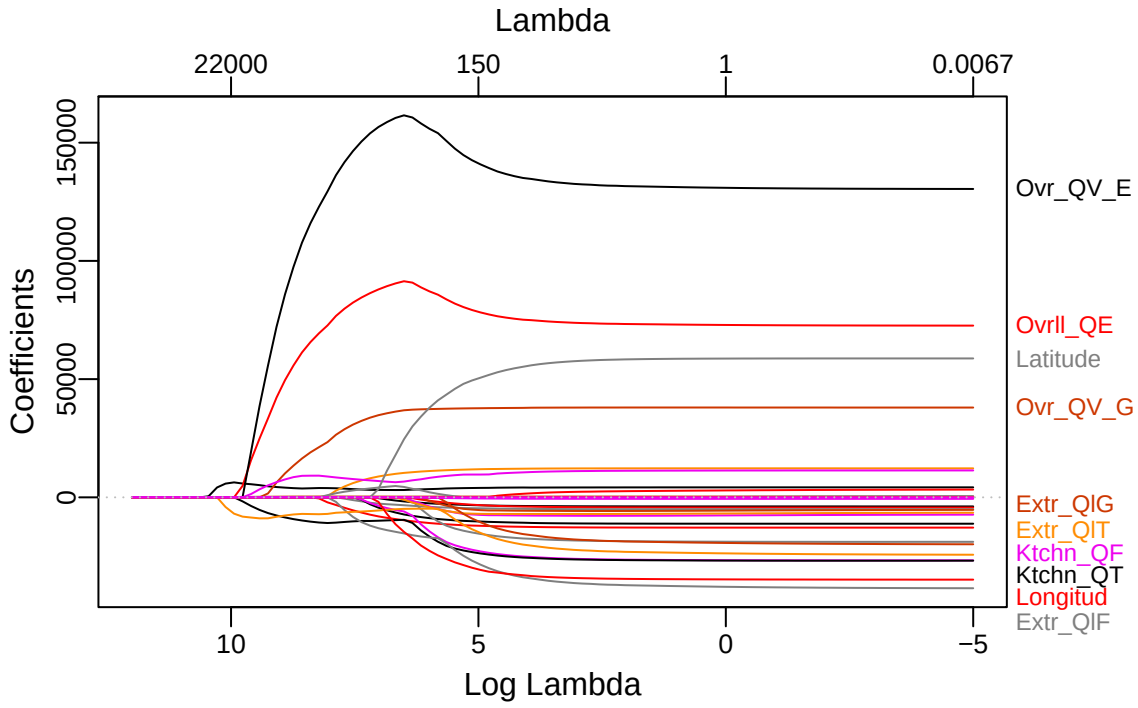
```
training_data <- read.csv("housing_training.csv") |> na.omit()
testing_data <- read.csv("housing_test.csv") |> na.omit()

x <- model.matrix(Sale_Price ~ ., training_data)[,-1]
y <- training_data[, "Sale_Price"]

x2 <- model.matrix(Sale_Price ~ .,testing_data)[, -1]
y2 <- testing_data$Sale_Price
```

## a)

```
set.seed(2026)
cv.lasso <- cv.glmnet(x, y,
                      alpha = 1,
                      lambda = exp(seq(12, -5, length = 100)))

plot(cv.lasso)
```




```
plot_glmnet(cv.lasso$glmnet.fit)
```




```
cv.lasso$lambda.1se
```

```
## [1] 1119.013
```

```
coef_1se <- predict(cv.lasso, s = "lambda.1se", type = "coefficients")
sum(coef_1se != 0) - 1
```

```
## [1] 29
```

```
lasso_pred <- predict(
  cv.lasso, newx = model.matrix(Sale_Price ~ ., testing_data)[,-1],
             s = "lambda.1se", type = "response")

lasso_test_RMSE = sqrt(mean((lasso_pred - testing_data$Sale_Price)^2))
lasso_test_RMSE
```

```
## [1] 20651.86
```

The selected tuning parameter is 1119.0126581. The test error is $2.0651861 \times 10^4$. Using 1SE rule, the number of predictors included is 29.

## b)

```
ctrl1 <- trainControl(method = "cv", number = 10)

set.seed(2026)
enet.fit <- train(Sale_Price ~ .,
                  data = training_data,
                  method = "glmnet",
                  tuneGrid = expand.grid(alpha = seq(0, 1, length = 21),
                                         lambda = exp(seq(12, -5, length = 100))),
                  trControl = ctrl1)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
enet.fit$bestTune
```

```
##     alpha   lambda
## 167  0.05 563.0302
```

```
enet.pred <- predict(enet.fit, newdata = testing_data)
enet_test_RMSE = sqrt(mean((enet.pred - testing_data[, "Sale_Price"])^2))
enet_test_RMSE
```

```
## [1] 20947.88
```

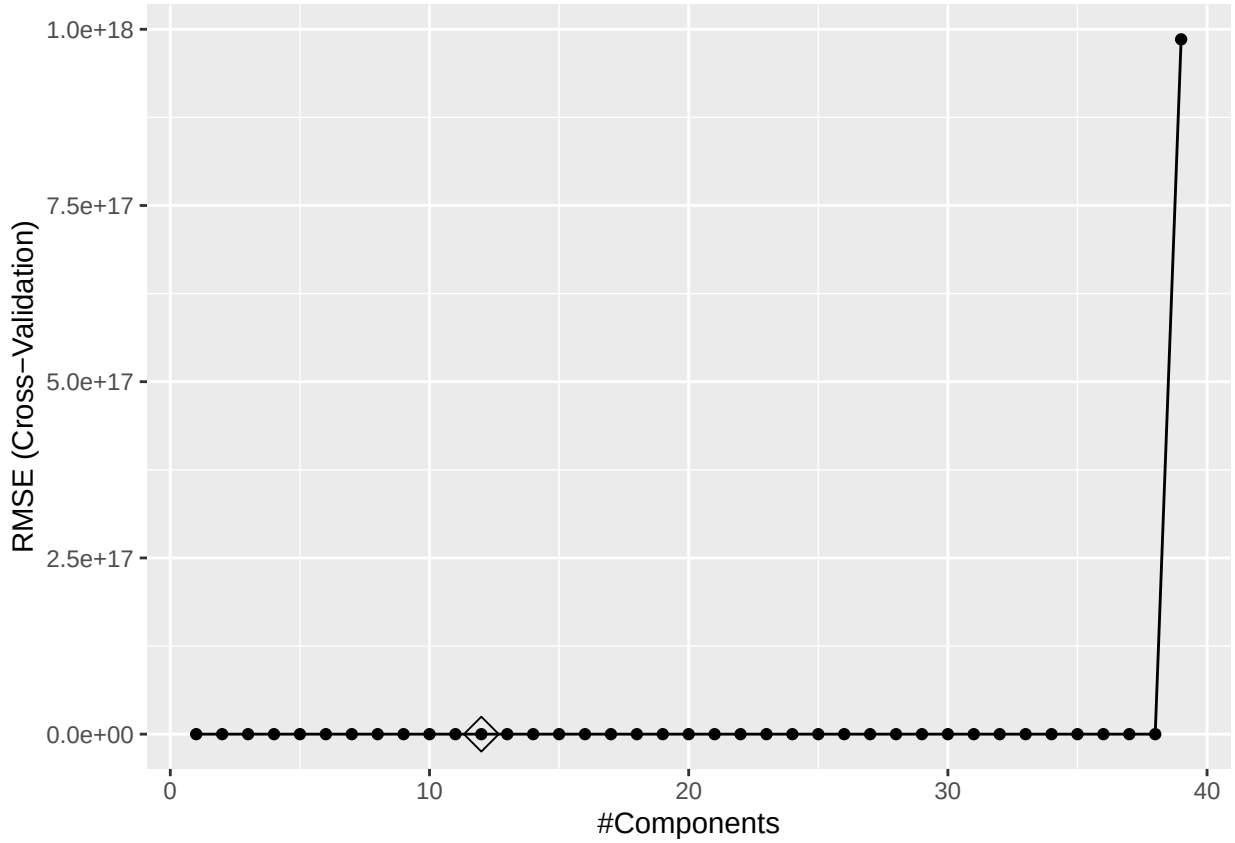
The selected tuning parameter is 0.05, 563.030236835952. The test error is $2.094788 \times 10^4$. 1SE is not applicable in elastic net because it has 2 tuning parameters, alpha and lambda. There is no way to apply one standard error of the minimum to 2 parameter. Therefore, 1SE is not applicable here.

## c)

```
set.seed(2026)
pls_fit <- train(x, y,
                 method = "pls",
                 tuneGrid = data.frame(ncomp = 1:ncol(x)),
                 trControl = ctrl1,
                 preProcess = c("center", "scale"))

predy2_pls2 <- predict(pls_fit, newdata = x2)
pls_test_RMSE = sqrt(mean((y2 - predy2_pls2)^2))
pls_test_RMSE
```

```
## [1] 21204.31
```

```
ggplot(pls_fit, highlight = TRUE)
```




```
pls_fit$bestTune
```

```
##    ncomp
## 12    12
```

The test error is $2.1204309 \times 10^4$. The number of components in the model is 12.

## d)

```
lasso_test_RMSE
```

```
## [1] 20651.86
```

```
enet_test_RMSE
```

```
## [1] 20947.88
```

```
pls_test_RMSE
```

```
## [1] 21204.31
```

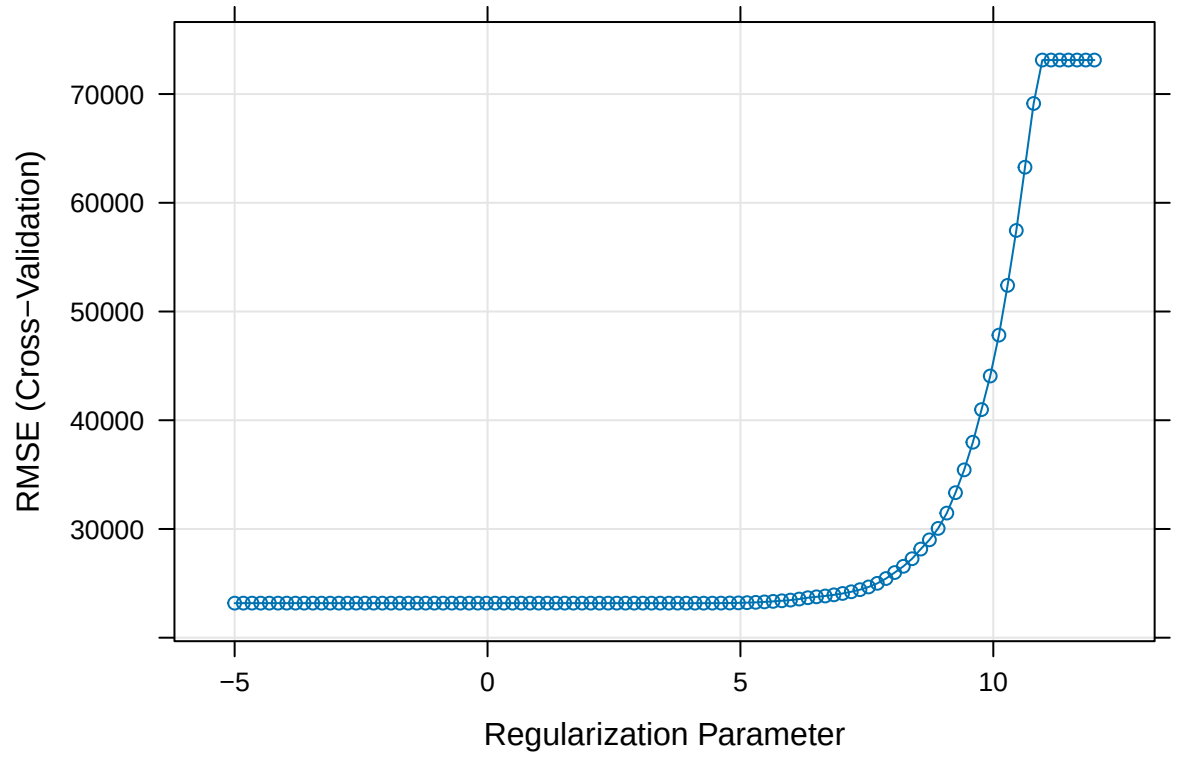
Among the three tests, the lasso model has the smallest RMSE: $2.0651861 \times 10^4$. Thus it is the best model.

e)

```
set.seed(2026)
lasso.fit <- train(Sale_Price ~ .,
                   data = training_data,
                   method = "glmnet",
                   tuneGrid = expand.grid(alpha = 1,
                                          lambda = exp(seq(12, -5, length = 100))),
                   trControl = ctrl1)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
plot(lasso.fit, xTrans = log)
```




```
lasso.fit$bestTune
```

```
##    alpha   lambda
## 54     1 60.40127
```

```
coef_caret_lasso = coef(lasso.fit$finalModel, lasso.fit$bestTune$lambda)
sum(coef_caret_lasso != 0) - 1
```

```
## [1] 37
```

```
lasso.pred.caret <- predict(lasso.fit, newdata = testing_data)
lasso_caret_RMSE = sqrt(mean((lasso.pred.caret - testing_data$Sale_Price)^2))
lasso_caret_RMSE
```

```
## [1] 20987.1
```

To have the lasso with glmnet in a) have comparable result to the caret method, check the result use lambda.min:

```
set.seed(2026)
cv.lasso$lambda.min
```

```
## [1] 71.71696
```

```
coef_min <- predict(cv.lasso, s = "lambda.min", type = "coefficients")
sum(coef_min != 0) - 1
```

```
## [1] 37
```

```
lasso_pred_min <- predict(
  cv.lasso, newx = model.matrix(Sale_Price ~ ., testing_data)[,-1],
             s = "lambda.min", type = "response")

lasso_min_test_RMSE = sqrt(mean((lasso_pred_min - testing_data$Sale_Price)^2))
lasso_min_test_RMSE
```

```
## [1] 20969.33
```

From the two tests (both using lambda.min), the number of predictors are the same: 37; The selected lambda are quite different with glmnet being 71.7169609 and caret being 1, 60.401268175901; the test RMSE are slightly different with glmnet being $2.0969327 \times 10^4$ and caret being $2.0987104 \times 10^4$. The possible reason behind such discrepancies is that the caret implementation did not specify lambda. It did not pass my specified lambda.grid, rather it used its built in default grid of lambda during cross validation. But the glmnet did use my lambda grid, so the two packages were run with different grid of lambda.